

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Full Stack Development Using Mern Stack

Practical 8 – Mini Project

Roll No.: T032	Name: KHUSHI
Class: TYIT	Batch: 2
Date of Assignment: 25-09-26	Date/Time of Submission: 25-09-26

Project Title : College Canteen Pre-Order System

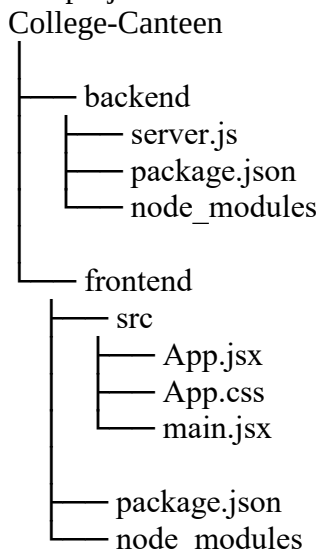
1. Aim - To develop a College Canteen Pre-Order System using the MERN Stack that allows students to view available food items, place food orders, and store and retrieve order details from MongoDB.

2. Technologies Used

Technology	Purpose
MongoDB	Stores student order data
Express.js	Creates backend APIs
React.js	Creates the user interface
Node.js	Runs the backend server
Axios	Connects React frontend with Express backend
CSS	Provides styling and responsive design
MongoDB Compass	Used to view stored orders

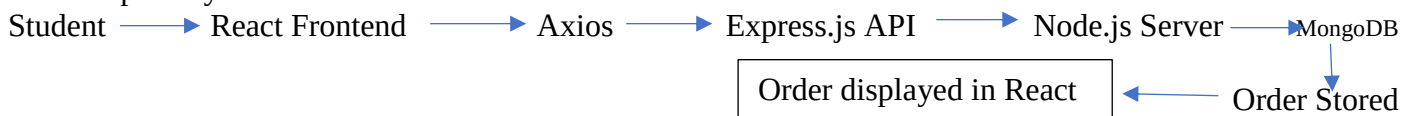
3. Project Structure

The project is divided into two main parts:



4. How the System Works

The complete system works as follows:



5. Backend Setup

Open the backend folder in VS Code.

Install the required packages:

`npm init -y`

`npm install express mongoose cors`

```

PS C:\Users\Lenovo\Downloads\MERN STACK> cd College-Canteen
PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen> mkdir backend

Directory: C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen

Mode                LastWriteTime         Length Name
----                -
d-----         25-09-2026     11:20         backend

PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen> cd backend
PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen\backend> npm init -y
PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen\backend> npm init -y
Wrote to C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen\backend\package.json:
{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen\backend> npm install express mongoose cors

added 88 packages, and audited 89 packages in 1m

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

6. Backend Code –

server.js

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
app.use(cors());
app.use(express.json());
/* MongoDB Connection */
mongoose.connect("mongodb://localhost:27017/canteenDB")
  .then(() => {
    console.log("MongoDB Connected");
  })
  .catch((error) => {
    console.log("MongoDB Error:", error);
  });
/* Food Items */
const foodItems = [
  { id: 1, name: "Veg Sandwich", price: 40 },
  { id: 2, name: "Masala Dosa", price: 60 },
  { id: 3, name: "Vada Pav", price: 25 },
  { id: 4, name: "Cold Coffee", price: 50 },
  { id: 5, name: "Pizza", price: 100 },
  { id: 6, name: "Burger", price: 80 },
  { id: 7, name: "French Fries", price: 60 },
  { id: 8, name: "Samosa", price: 20 },
  { id: 9, name: "Pav Bhaji", price: 70 },
  { id: 10, name: "Paneer Roll", price: 75 },
  { id: 11, name: "Cheese Sandwich", price: 65 },
  { id: 12, name: "Idli Sambar", price: 50 },

```

```

    { id: 13, name: "Poha", price: 35 },
    { id: 14, name: "Maggie", price: 45 },
    { id: 15, name: "Tea", price: 15 },
    { id: 16, name: "Lemon Juice", price: 30 },
    { id: 17, name: "Chocolate Milkshake", price: 70 },
    { id: 18, name: "Veg Momos", price: 60 },
    { id: 19, name: "Noodles", price: 65 },
    { id: 20, name: "Chole Bhature", price: 80 }
  ];
  /* Order Schema */
  const orderSchema = new mongoose.Schema({
    name: String,
    rollNo: String,
    course: String,
    foodItem: String,
    quantity: Number,
    totalPrice: Number
  });
  const Order = mongoose.model("Order", orderSchema);
  /* GET /food */
  app.get("/food", (req, res) => {
    res.json(foodItems);
  });
  /* POST /orders */
  app.post("/orders", async (req, res) => {
    try {
      const order = new Order(req.body);
      await order.save();
      res.json({
        message: "Order placed successfully",
        order: order
      });
    } catch (error) {
      res.status(500).json({
        error: error.message
      });
    }
  });
  /* GET /orders */
  app.get("/orders", async (req, res) => {
    try {
      const orders = await Order.find();
      res.json(orders);
    } catch (error) {
      res.status(500).json({
        error: error.message
      });
    }
  });
  /* Start Server */
  app.listen(5000, () => {
    console.log("Server running on port 5000");
  });

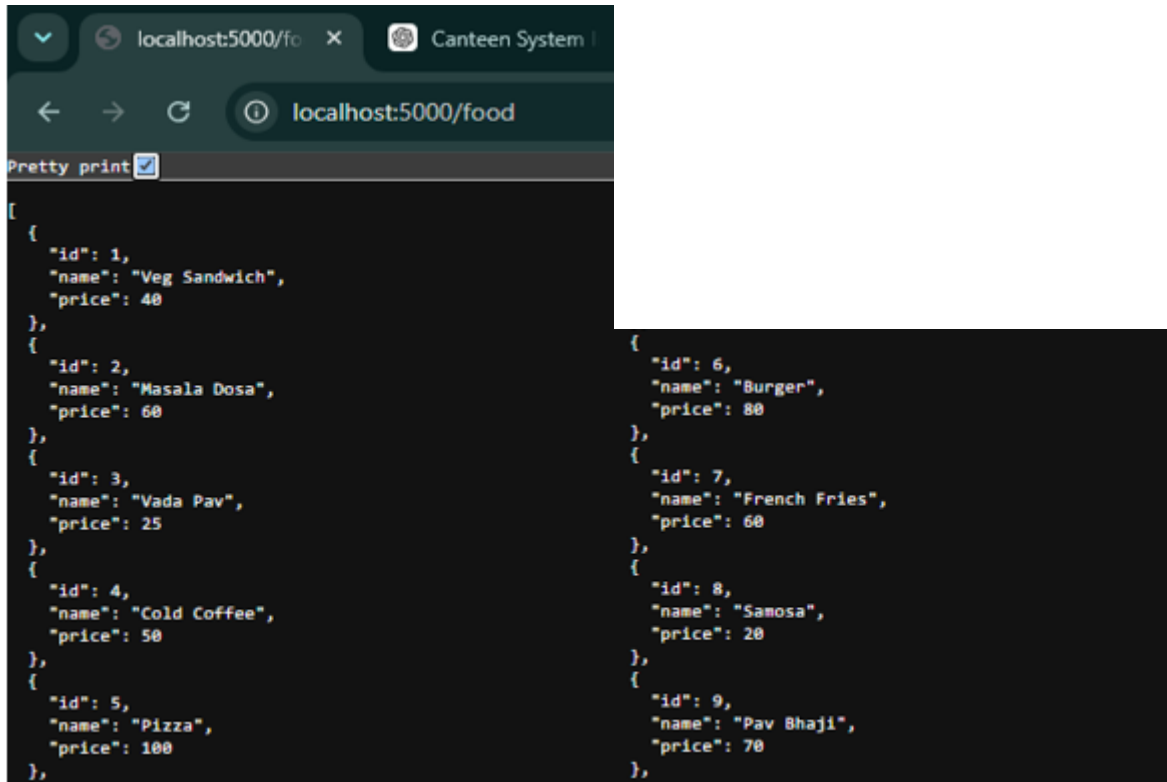
```

7. API 1 – Get Food Items

```
app.get("/food", (req, res) => {
  res.json(foodItems);
});
```

Purpose - This API retrieves all available food items.

URL - <http://localhost:5000/food>



8. Explanation of Backend Code

Express

```
const express = require("express");
```

Express is used to create the backend server and APIs.

MongoDB

```
mongoose.connect("mongodb://localhost:27017/canteenDB")
```

This connects the application to the MongoDB database named: canteenDB

CORS

```
app.use(cors());
```

CORS allows the React frontend to communicate with the backend.

JSON

```
app.use(express.json());
```

This allows Express to receive JSON data from React.

9. API 2 – Place Order

```
app.post("/orders", async (req, res) => {
  const order = new Order(req.body);
  await order.save();
  res.json({
    message: "Order placed successfully",
    order: order
  });
});
```

Purpose

This API receives the student's order and stores it in MongoDB.

Example data:

```
{
  "name": "Ravi",
  "rollNo": "102",
  "course": "BSc IT",
  "foodItem": "Pav Bhaji",
  "quantity": 2,
  "totalPrice": 140
}
```

10. API 3 – Get Orders

```
app.get("/orders", async (req, res) => {
  const orders = await Order.find();
  res.json(orders);
});
```

Purpose

This API retrieves all orders stored in MongoDB.

URL

<http://localhost:5000/orders>

11. Frontend Setup

Create the React application using Vite:

```
npm create vite@latest frontend -- --template react
```

Then:

```
cd frontend
```

Install dependencies:

```
npm install
```

Install Axios:

```
npm install axios
```

Run the frontend:

```
npm run dev
```

The application runs at:

<http://localhost:5173/>

12. React Code –

App.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";
function App() {
  const [food, setFood] = useState([]);
  const [orders, setOrders] = useState([]);
  const [form, setForm] = useState({
    name: "",
    rollNo: "",
    course: "",
    foodItem: "",
    quantity: 1
  });
  /* Load food and orders */
  useEffect(() => {
    const loadData = async () => {
      try {
        const foodResponse =
```

```

    await axios.get("http://localhost:5000/food");
    const orderResponse =
    await axios.get("http://localhost:5000/orders");
    setFood(foodResponse.data);
    setOrders(orderResponse.data);
  } catch (error) {
    console.log("Error loading data:", error);
  }
};
loadData();
}, []);
/* Handle form input */
const handleChange = (event) => {
  setForm({
    ...form,
    [event.target.name]: event.target.value
  });
};
/* Place Order */
const placeOrder = async (event) => {
  event.preventDefault();
  const selectedFood = food.find(
    (item) => item.name === form.foodItem
  );
  if (!selectedFood) {
    alert("Please select a food item.");
    return;
  }
  const orderData = {
    name: form.name,
    rollNo: form.rollNo,
    course: form.course,
    foodItem: form.foodItem,
    quantity: Number(form.quantity),
    totalPrice:
      selectedFood.price * Number(form.quantity)
  };
  try {
    await axios.post(
      "http://localhost:5000/orders",
      orderData
    );
    alert("Order placed successfully!");
    setForm({
      name: "",
      rollNo: "",
      course: "",
      foodItem: "",
      quantity: 1
    });
    const response =
      await axios.get("http://localhost:5000/orders");
    setOrders(response.data);
  } catch (error) {
    console.log("Error placing order:", error);
  }
};

```

```

    alert("Failed to place order.");
} };
return (
  <div className="container">
    <h1>🍽️ College Canteen Pre-Order System</h1>
    <h2>Available Food Items</h2>
    <div className="food-list">
      {food.map((item) => (
        <div
          className="food-card"
          key={item.id}
        >
          <h3>{item.name}</h3>
          <p>₹ {item.price}</p>
        </div>
      ))}
    </div>
    <h2>Student Order Form</h2>
    <form onSubmit={placeOrder}>
      <label>Student Name</label>
      <input
        type="text"
        name="name"
        placeholder="Enter student name"
        value={form.name}
        onChange={handleChange}
        required
      />
      <label>Roll Number</label>
      <input
        type="text"
        name="rollNo"
        placeholder="Enter roll number"
        value={form.rollNo}
        onChange={handleChange}
        required
      />
      <label>Course</label>
      <input
        type="text"
        name="course"
        placeholder="Enter course"
        value={form.course}
        onChange={handleChange}
        required
      />
      <label>Food Item</label>
      <select
        name="foodItem"
        value={form.foodItem}
        onChange={handleChange}
        required
      >
        <option value="">
          Select Food Item

```

```

</option>
{food.map((item) => (
  <option
    key={item.id}
    value={item.name}
  >
    {item.name} - ₹ {item.price}
  </option>
))}
</select>
<label>Quantity</label>
<input
  type="number"
  name="quantity"
  min="1"
  value={form.quantity}
  onChange={handleChange}
  required
/>
<button type="submit">
  Place Order
</button>
</form>
<h2>Placed Orders</h2>
{orders.length === 0 ? (
  <p className="no-orders">
    No orders placed yet.
  </p>
) : (
  <table>
    <thead>
      <tr>
        <th>Name</th>
        <th>Roll No</th>
        <th>Course</th>
        <th>Food</th>
        <th>Quantity</th>
        <th>Total</th>
      </tr>
    </thead>
    <tbody>
      {orders.map((order) => (
        <tr key={order._id}>
          <td>{order.name}</td>
          <td>{order.rollNo}</td>
          <td>{order.course}</td>
          <td>{order.foodItem}</td>
          <td>{order.quantity}</td>
          <td>₹ {order.totalPrice}</td>
        </tr>
      ))}
    </tbody>
  </table>
)}
</div>

```

```
);  
}  
export default App;
```

13. CSS – App.css

```
// COLLEGE CANTEEN PRE-ORDER SYSTEM Professional Peach Theme  
// GLOBAL
```

```
* {  
  box-sizing: border-box;  
}
```

```
body {  
  margin: 0;  
  font-family: "Segoe UI", Roboto, Arial, sans-serif;  
  background: #fff7f2;  
  color: #33221d;  
  line-height: 1.5;  
}
```

```
// MAIN CONTAINER
```

```
.container {  
  width: 92%;  
  max-width: 1150px;  
  margin: 40px auto;  
  padding: 40px;  
  background: #ffffff;  
  border: 1px solid #f1ddd4;  
  border-radius: 16px;  
  box-shadow: 0 8px 30px rgba(75, 45, 35, 0.08);  
}
```

```
// MAIN TITLE
```

```
h1 {  
  margin: 0 0 45px;  
  text-align: center;  
  color: #9d4f35;  
  font-size: 34px;  
  font-weight: 700;  
  letter-spacing: -0.5px;  
}
```

```
// SECTION HEADINGS
```

```
h2 {  
  margin: 42px 0 20px;  
  padding-bottom: 10px;  
  color: #63382c;  
  font-size: 23px;  
  font-weight: 650;  
  border-bottom: 2px solid #f3d3c4;  
}
```

```
// FOOD SECTION
```

```
.food-list {  
  display: grid;
```

```
grid-template-columns: repeat(4, 1fr);
gap: 18px;
}
.food-card {
padding: 22px 18px;
background: #fffaf7;
border: 1px solid #efd9cf;
border-radius: 12px;
text-align: center;
transition: all 0.2s ease;
}
.food-card:hover {
transform: translateY(-3px);
border-color: #df9b80;
box-shadow: 0 8px 20px rgba(157, 79, 53, 0.12);
}
.food-card h3 {
margin: 0 0 12px;
color: #55342b;
font-size: 16px;
font-weight: 600;
}
.food-card p {
margin: 0;
color: #c35f3d;
font-size: 19px;
font-weight: 700;
}

// ORDER FORM
form {
max-width: 760px;
margin: 0 auto;
padding: 30px;
background: #fffaf7;
border: 1px solid #efd9cf;
border-radius: 14px;
box-shadow: 0 5px 18px rgba(75, 45, 35, 0.06);
}
label {
display: block;
margin: 16px 0 7px;
color: #63382c;
font-size: 14px;
font-weight: 600;
}
input,
select {
width: 100%;
padding: 12px 14px;
margin-bottom: 5px;
border: 1px solid #dfc3b7;
border-radius: 8px;
background: #ffffff;
color: #33221d;
```

```
font-family: inherit;
font-size: 15px;
outline: none;
transition: border-color 0.2s, box-shadow 0.2s;
}
input::placeholder {
  color: #aa9188;
}
input:hover,
select:hover {
  border-color: #c98970;
}
input:focus,
select:focus {
  border-color: #c35f3d;
  box-shadow: 0 0 0 3px rgba(195, 95, 61, 0.10);
}

// BUTTON
button {
  display: block;
  width: 190px;
  margin: 25px auto 0;
  padding: 12px 20px;
  border: none;
  border-radius: 8px;
  background: #b85c3d;
  color: #ffffff;
  font-family: inherit;
  font-size: 15px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s ease;
}
button:hover {
  background: #98472f;
  transform: translateY(-1px);
  box-shadow: 0 5px 14px rgba(152, 71, 47, 0.20);
}
button:active {
  transform: translateY(0);
}

// ORDERS TABLE
table {
  width: 100%;
  margin-top: 20px;
  border-collapse: collapse;
  overflow: hidden;
  background: #ffffff;
  border: 1px solid #ead5cc;
  border-radius: 10px;
  box-shadow: 0 4px 15px rgba(75, 45, 35, 0.05);
}
th {
```

```
padding: 14px 12px;
background: #a95338;
color: #ffffff;
text-align: center;
font-size: 14px;
font-weight: 600;
white-space: nowrap;
}
td {
padding: 13px 12px;
border-bottom: 1px solid #f0e1db;
color: #4d342d;
text-align: center;
font-size: 14px;
}
tbody tr:nth-child(even) {
background: #fffaf7;
}
tbody tr:hover {
background: #fff0e8;
}
tbody tr:last-child td {
border-bottom: none;
}

// NO ORDERS
.no-orders {
padding: 20px;
background: #fffaf7;
border: 1px solid #efd9cf;
border-radius: 10px;
color: #80665d;
text-align: center;
}

// RESPONSIVE DESIGN
@media (max-width: 900px) {
.food-list {
grid-template-columns: repeat(3, 1fr);
}
.container {
width: 94%;
padding: 30px;
}
}
@media (max-width: 650px) {
.container {
width: 94%;
margin: 20px auto;
padding: 22px;
}
h1 {
font-size: 27px;
margin-bottom: 35px;
}
```

```

h2 {
  font-size: 20px;
  margin-top: 32px;
}
.food-list {
  grid-template-columns: repeat(2, 1fr);
  gap: 12px;
}
.food-card {
  padding: 17px 10px;
}
form {
  padding: 20px;
}
table {
  display: block;
  overflow-x: auto;
  white-space: nowrap;
}
th,
td {
  padding: 10px 8px;
}
}
@media (max-width: 400px) {
  .food-list {
    grid-template-columns: 1fr;
  }
  h1 {
    font-size: 24px;
  }
}

```

14. How Axios is Used

Axios connects React with the Express backend.

Get food:

```
axios.get("http://localhost:5000/food")
```

Get orders:

```
axios.get("http://localhost:5000/orders")
```

Place an order:

```

axios.post(
  "http://localhost:5000/orders",
  orderData
)

```

15. How MongoDB is Used

MongoDB stores the orders permanently.

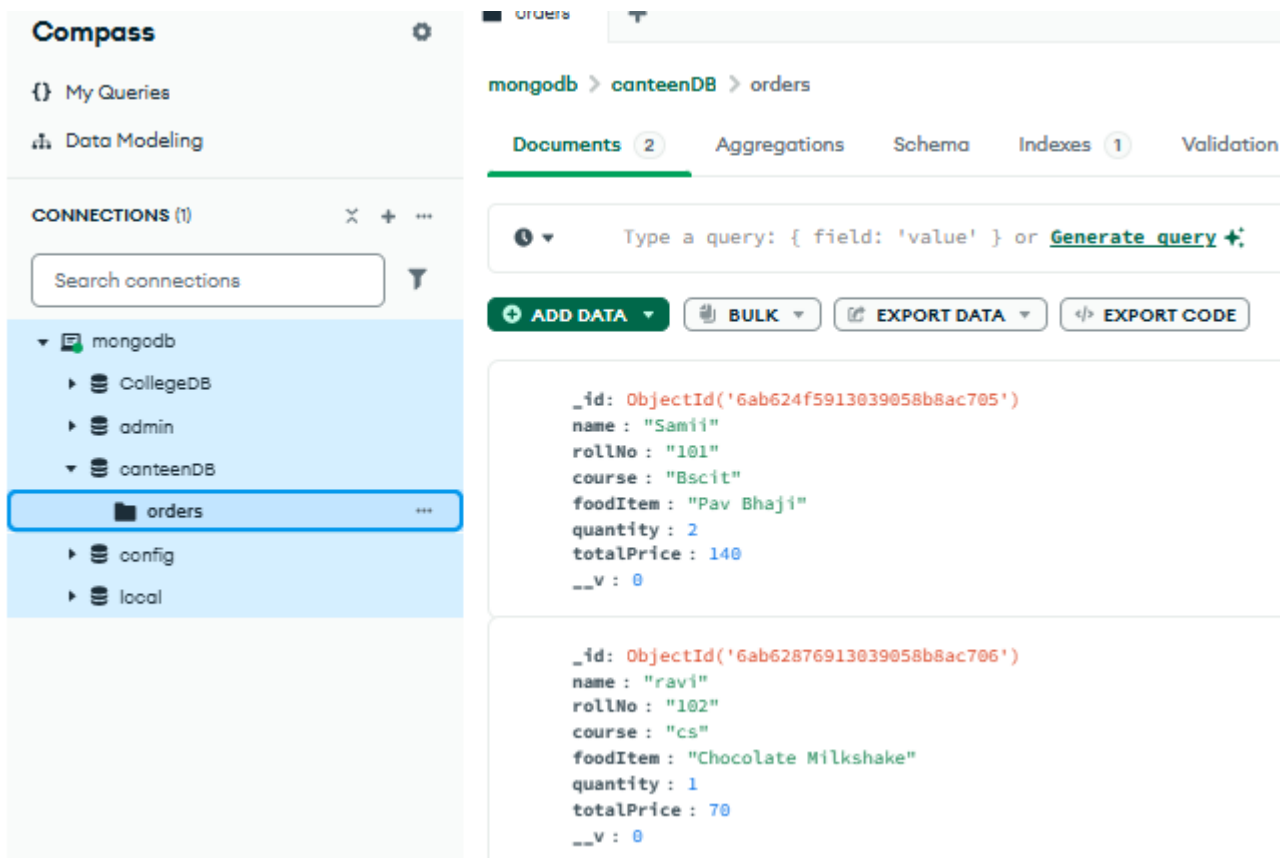
Database:

canteenDB

Collection:

orders

Example stored document:



MongoDB Compass can be used to view these documents.

16. Running the Project

Start MongoDB

Make sure MongoDB is running.

Start Backend

Open terminal in:

College-Canteen/backend

Run:

node server.js

Expected output:

```

PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen\backend> node server.js
Server running on port 5000
MongoDB Connected

```

Start Frontend

Open another terminal in:

cd College-Canteen

npm create vite@latest frontend -- --template react

Run:

npm run dev

Expected output:

Local: http://localhost:5173/

Open the URL in Chrome.

```

PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen\frontend> npm install

up to date, audited 144 packages in 4s

31 packages are looking for funding
run `npm fund` for details

```

```

PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen\frontend> npm install axios

added 25 packages, and audited 169 packages in 4s

37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```

```

PS C:\Users\Lenovo\Downloads\MERN STACK\College-Canteen\frontend> npm run dev

> frontend@0.0.0 dev
> vite

11:52:21 am [vite] (client) Re-optimizing dependencies because lockfile has changed

VITE v8.3.1 ready in 770 ms

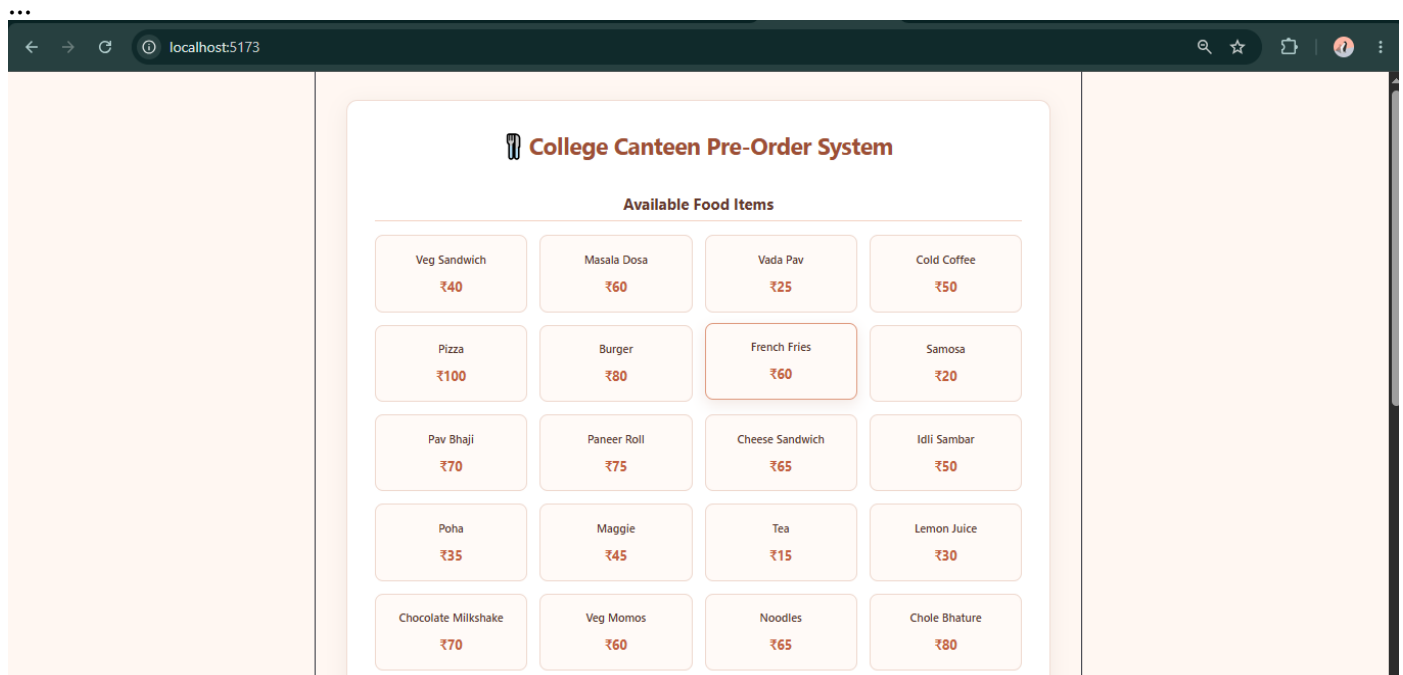
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
11:54:40 am [vite] (client) hmr update /src/App.jsx
11:54:50 am [vite] (client) [optimizer] bundling dependencies...
11:54:50 am [vite] (client) dependency optimized: axios
11:54:50 am [vite] (client) optimized dependencies changed. reloading
11:55:23 am [vite] (client) hmr update /src/App.css
11:56:32 am [vite] (client) hmr update /src/App.jsx

```

17. Output 1 – Available Food Items

The React page displays available food items such as:

Veg Sandwich	₹40
Masala Dosa	₹60
Vada Pav	₹25
Cold Coffee	₹50
Pizza	₹100
Burger	₹80
French Fries	₹60
Samosa	₹20
Pav Bhaji	₹70
Paneer Roll	₹75



Student Order Form

Student Name

Shree

Roll Number

001

Course

Bscit

Food Item

French Fries - ₹60

Quantity

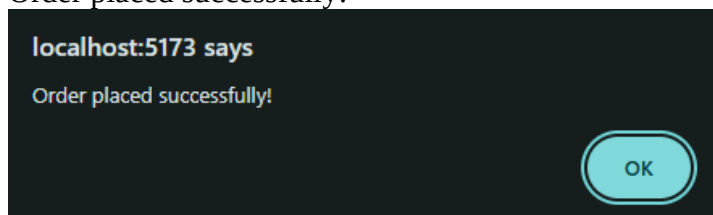
5

Place Order

Order form filled before clicking **Place Order**.

19. Output 3 – Successful Order

After clicking **Place Order**, the system displays:
Order placed successfully!



Placed Orders

Name	Roll No	Course	Food	Quantity	Total
Samii	101	Bscit	Pav Bhaji	2	₹140
ravi	102	cs	Chocolate Milkshake	1	₹70
Shree	001	Bscit	French Fries	5	₹300

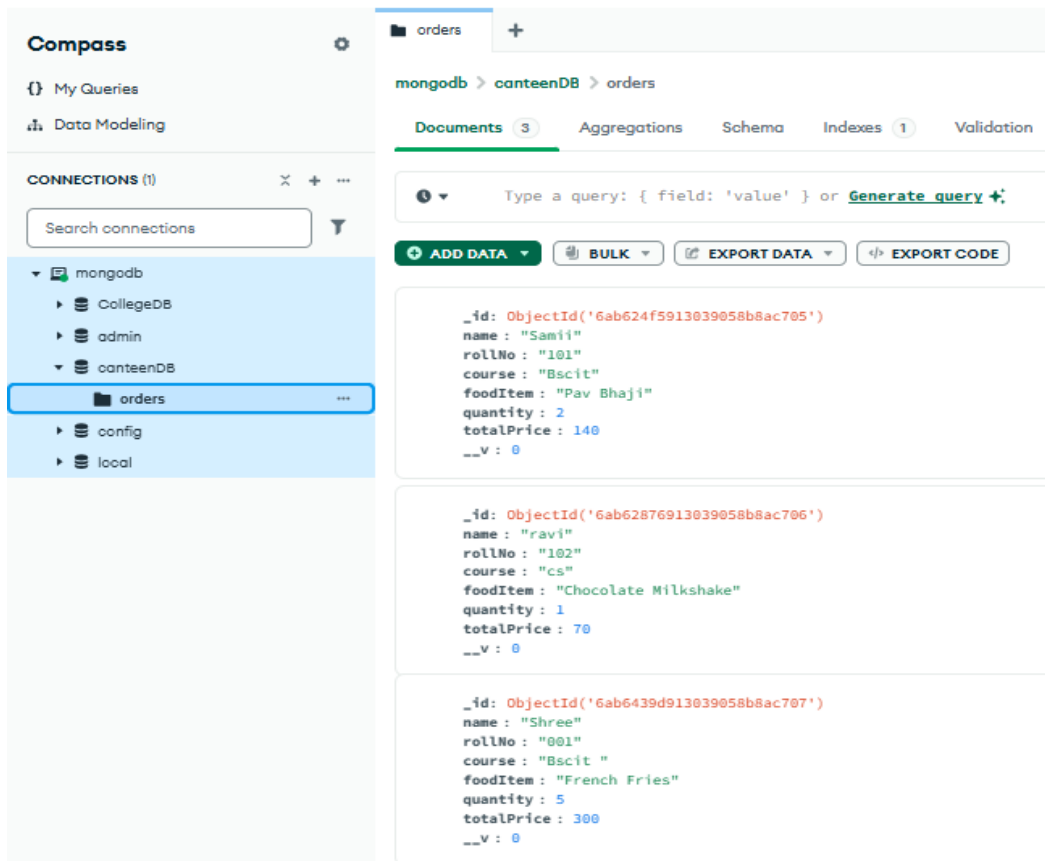
The order then appears in the **Placed Orders** table.

20. Output 4 – MongoDB Compass

Open:

MongoDB Compass → **canteenDB** → **orders** → **Documents**

The stored order appears as:

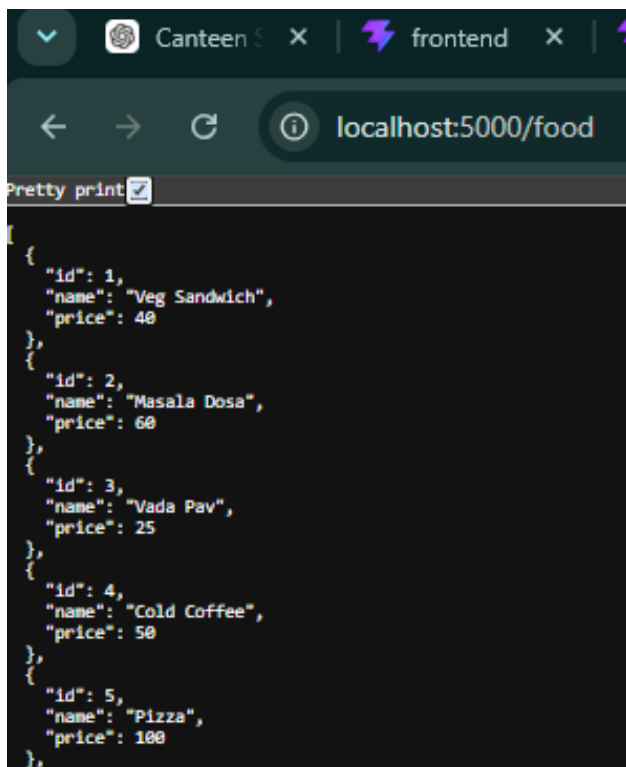


Your MongoDB orders collection.

21. Output 5 – GET Food API

Open in browser:

<http://localhost:5000/food>



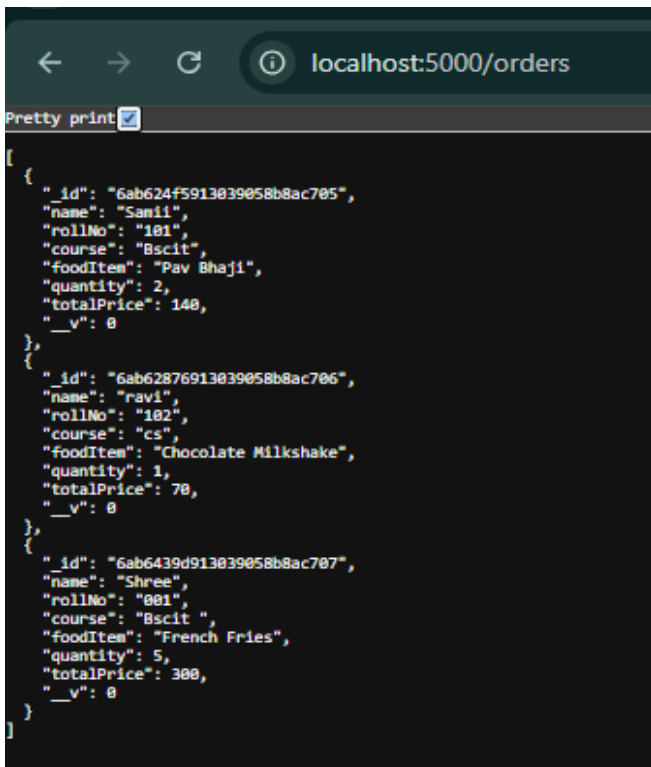
Browser displaying the JSON food data.

22. Output 6 – GET Orders API

Open:

<http://localhost:5000/orders>

It displays the orders stored in MongoDB.



Browser showing the /orders JSON response.

23. Advantages

1. Students can view available food items easily.
2. Students can place orders before reaching the canteen.
3. Order information is stored in MongoDB.
4. Reduces manual order management.
5. Provides a simple and user-friendly interface.
6. Uses a complete MERN Stack architecture.
7. The application is responsive and can be viewed on mobile devices.

24. Conclusion

The **College Canteen Pre-Order System** was successfully developed using the **MERN Stack**. The system allows students to view food items, enter their details, select food and quantity, and place orders. The orders are sent through Axios to the Express/Node.js backend and stored in MongoDB. The stored orders can then be retrieved and displayed on the React frontend.

Thus, the project demonstrates the practical implementation of **React, Node.js, Express.js, MongoDB, Mongoose, and Axios** in a full-stack web application.
